

Branching and boundary tests

Project: Python readiness portfolio

1. Your mission

Your learning target

I can trace the selected branch in an if elif else chain.

I can use indentation to identify which statements belong to a branch.

I can test values at and around a decision boundary.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Boolean conditions. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Trace the selected branch in an if elif else chain

Use indentation to identify which statements belong to a branch

Test values at and around a decision boundary

Retrieve what you already know

Does minutes ≤ 12 accept exactly 12? How does $<$ differ?

My starting explanation

This week's end goal

Trace branching and boundary tests and explain the changed program before running it.

2. Notice the evidence

A fictional activity sorter labels a supplied integer duration. Negative values are Invalid; values from zero through 12 are Short; larger values are Long. The labels describe duration only. Write this requirement before the program so that a test has an expected answer independent of the code.

An if statement selects a block when its condition is true. An elif is considered only if earlier conditions in the same chain were false. An else handles the remaining cases. Colons end the condition lines. Use four spaces for the indented statements and keep related branches aligned.

List the given facts and one fact that is not supplied.

3. See how it works

One chain chooses one label

Input · Checks in order · Selected label

Input: -1

Checks in order: Below zero is True

Selected label: Invalid

Input: 12

Checks in order: Below zero False; <= 12 True

Selected label: Short

Input: 13

Checks in order: Both checks False

Selected label: Long

Later branches in this chain are skipped after a condition succeeds.

```
minutes = 12

if minutes < 0:
    label = "Invalid"

elif minutes <= 12:
    label = "Short"

else:
    label = "Long"

print(label)
```

The output is Short. For 12, the first condition is false and the second is true. The else block is skipped. `print(label)` is not indented, so it runs after the selected branch. Each path assigns label before this line reads it. Change only minutes and save before each fresh run.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

For -1, 12 and 13, record checked conditions, skipped conditions and the final label. Run each as a fresh script after predicting.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: Remove the first negative-count branch.

Prediction: A count of -2 would incorrectly receive small.

Reason: Negative two is less than five, but it is not a permitted measured count.

5. Words that unlock the idea

Word · Meaning

Word: Branch

Meaning: A possible path through a program

Word: Condition

Meaning: An expression used to decide which path to take

Word: Indentation

Meaning: Leading spaces that identify a Python code block

Word: Boundary

Meaning: A value where a classification may change

Word: Regression check

Meaning: A previous test rerun after an edit to detect a new failure

Use two terms to explain a step in this week's example.

6. Follow a worked example

Input minutes · Expected label · Reason for this test

Input minutes: -1

Expected label: Invalid

Reason for this test: Negative input rejection

Input minutes: 0

Expected label: Short

Reason for this test: Lowest valid duration

Input minutes: 11

Expected label: Short

Reason for this test: Just below the upper boundary

Input minutes: 12

Expected label: Short

Reason for this test: Exact upper boundary

Input minutes: 13

Expected label: Long

Reason for this test: Just above the boundary

These tests exercise every branch and the equality case. A change from ≤ 12 to < 12 is exposed by 12; testing 11 alone misses it. Branch coverage means every branch was exercised, but it does not prove every condition is correct.

Suppose the program uses < 12 instead of ≤ 12 . What is printed for 12, why does that violate the written rule, and which earlier test should still pass after repair?

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Branching and boundary tests

Check invalid values before branching. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

The facts we will use

Python 3: `if count<0: label="invalid"; elif count<5: label="small"; else: label="large"`. Read as indented branches.

AI ACADEMY / PYTHON-BRIDGE/5

Branching and boundary tests

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Check validity

For count=-2, the first condition selects invalid.



2. Check small boundary

For count=4, validity passes and count<5 selects small.



3. Handle equality

For count=5, both earlier conditions fail, so large is selected.

CHANGE ONE THING

Remove the first negative-count branch.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Check validity

What happens and why: For count=-2, the first condition selects invalid.

Step: 2. Check small boundary

What happens and why: For count=4, validity passes and count<5 selects small.

Step: 3. Handle equality

What happens and why: For count=5, both earlier conditions fail, so large is selected.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

An if/elif/else chain selects the first matching branch. Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

What this example does not establish

The toy procedure assumes numeric counts; validate types and define missing-value behaviour before using it in a real service.

7. Find and repair a mistake

```
minutes = -1

if minutes < 0:
    label = "Invalid"

if minutes <= 12:
    label = "Short"

else:
    label = "Long"

print(label)
```

This faulty version prints Short. The second if starts a separate decision and overwrites Invalid. Its else belongs to that second if. Restoring elif makes the second test depend on the first test failing. Moving print inside one branch can also hide output on other paths; indentation affects behavior, not just appearance.

The script rejects negative integers, but it does not validate arbitrary types or check a measured duration. A supplied text value can raise an error during comparison. Keep the stated integer input contract and report the limitation honestly.

Locate the earliest unsupported step

Trace the two-independent-if example for -1. Identify which assignment overwrites the earlier label and make the smallest repair.

State one repair and the checks you would rerun.

8. Practise together

Describe the purpose of if, elif and else in the worked duration sorter. Explain why print(label) is outside the indented branches.

Suppose the program uses < 12 instead of ≤ 12 . What is printed for 12, why does that violate the written rule, and which earlier test should still pass after repair?

Explain your prediction to a partner before comparing results.

Practise with less help: Trace the tested path

Use workbook W2 for the connected example “Branching and boundary tests”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Branching and boundary tests. Proposed explanation: “Every condition in the chain always runs”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Trace -2, 4 and 5 through the branches. Explain why the negative check comes first and what label -2 gets if it is removed.

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

For -1, 12 and 13, record checked conditions, skipped conditions and the final label. Run each as a fresh script after predicting.

```
minutes = 12

if minutes < 0:
    label = "Invalid"

elif minutes <= 12:
    label = "Short"

else:
    label = "Long"

print(label)
```

```
minutes = -1

if minutes < 0:
    label = "Invalid"

if minutes <= 12:
    label = "Short"

else:
    label = "Long"

print(label)
```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

Describe the purpose of if, elif and else in the worked duration sorter. Explain why print(label) is outside the indented branches.

Trace the two-independent-if example for -1. Identify which assignment overwrites the earlier label and make the smallest repair.

A repair changes the small-count condition from < 6 to ≤ 6 . Name the case that exposes the original error and two different kinds of earlier cases to rerun.

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled branching and boundary tests evidence sheet to Python readiness portfolio. Show how you can trace the selected branch in an if elif else chain. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week’s record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record: _____

Project evidence: Inputs permitted at prediction time

Record: _____

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

12. Show what you know

For each run, record input, expected label, conditions actually checked and observed label. Mark skipped conditions as skipped rather than pretending the program evaluated them. After a repair, rerun earlier cases as well as the failing one. Week 6 will repeat a block and track changes across iterations.

A learner tests -1, 2 and 20 and concludes the sorter is completely correct because all three branches ran. Explain what important case is missing and why branch coverage alone is insufficient.

A repair changes the small-count condition from < 6 to ≤ 6 . Name the case that exposes the original error and two different kinds of earlier cases to rerun.

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

One chain chooses one label

Input · Checks in order · Selected label

Input: -1

Checks in order: Below zero is True

Selected label: Invalid

Input: 12

Checks in order: Below zero False; <= 12 True

Selected label: Short

Input: 13

Checks in order: Both checks False

Selected label: Long

Later branches in this chain are skipped after a condition succeeds.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Trace branch order

Predict the action for 30, then trace 29 and -1. Explain why validity is checked before the lamp decision.

```
reading = 30
if not 0 <= reading <= 100:
    action = "invalid"
elif reading < 30:
    action = "on"
else:
    action = "off"
print(action)
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Use an integer tiles count. Label a negative value Invalid, zero Empty, 1 through 6 Small and values above 6 Large. Write an ordered if elif else script and predict -1, 0, 1, 6 and 7. Include one print after the chain.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.